

KubeQuest — Architecture du cluster : rôle de chaque nœud (support de présentation)

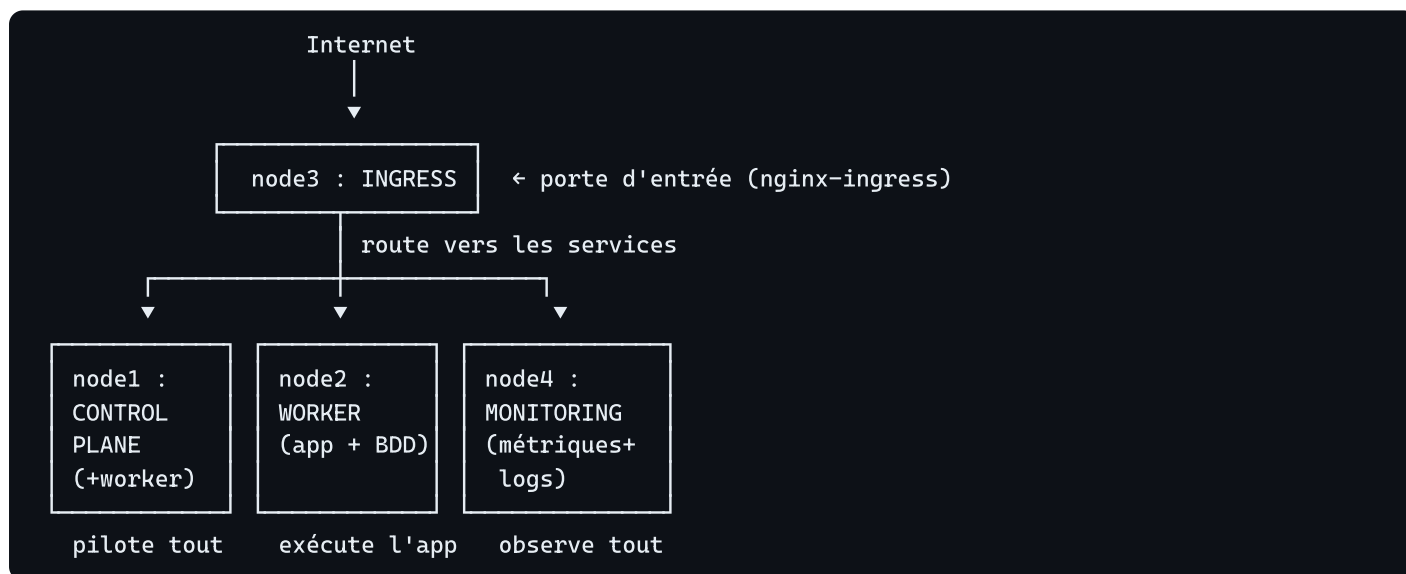
Cluster Kubernetes **kubeadm** (non managé) sur **AWS EC2**, région **eu-central-1**. 4 machines **t4g.medium** (ARM64 / Graviton, 2 vCPU, 4 Go RAM), OS **Amazon Linux 2023**. Kubernetes **v1.31.14**, runtime **containerd 2.2.1**, réseau (CNI) **Calico**.

1. Vue d'ensemble : pourquoi 4 nœuds avec des rôles séparés ?

Le sujet impose 4 VM avec une **séparation des responsabilités** (separation of concerns). Ce découpage n'est pas cosmétique, il répond à 3 objectifs d'ingénierie :

1. **Isolation des charges** : le trafic entrant (ingress), l'observabilité (monitoring) et les applications ne se disputent pas les mêmes ressources CPU/RAM. Si l'app monte en charge, elle ne fait pas tomber Prometheus, et inversement.
2. **Sécurité / surface d'attaque** : le nœud qui reçoit le trafic Internet (ingress) est distinct du plan de contrôle (le « cerveau » du cluster). Une compromission de l'entrée ne donne pas directement les clés du cluster.
3. **Lisibilité opérationnelle** : on sait où regarder. Un problème réseau → nœud ingress ; un dashboard qui rame → nœud monitoring ; l'API qui ne répond pas → control-plane.

Le placement des pods est piloté par des **labels de rôle** (`node-role.kubernetes.io/worker`, `.../ingress`, `.../monitoring`) et des **nodeSelector** / règles d'affinité dans les manifestes.



2. node1 — Control Plane (le cerveau)

Attribut	Valeur
Rôle	control-plane (+ premier worker)
IP publique	35.156.165.130 (Elastic IP = fixe)
IP privée	10.2.50.23
Instance	i-02dfc53b800341c20

Ce qui tourne dessus (composants système Kubernetes) : - **etcd** : la base de données clé-valeur qui stocke TOUT l'état du cluster (objets, config, secrets). C'est la mémoire du cluster. - **kube-apiserver** : le point d'entrée unique de

l'API Kubernetes. Chaque `kubectl` passe par lui. - `kube-controller-manager` : fait converger l'état réel vers l'état désiré (recrée les pods manquants, etc.). - `kube-scheduler` : décide sur quel nœud placer chaque nouveau pod. - **Calico** + `kube-proxy` : réseau des pods et routage des services.

Pourquoi ce choix : le control-plane est vital et sensible. On lui donne l'**Elastic IP fixe** (35.156.165.130) car c'est le point d'administration stable : toute la gestion (`kubectl`) passe par lui, et il ne doit pas « bouger » aux redémarrages. Conformément au sujet (« kube-1 = control plane + premier worker »), il peut aussi exécuter des charges, mais on garde les applications métier sur node2 pour ne pas risquer l'API.

Point de démo : « Toute l'administration du cluster se fait depuis node1 via `kubectl` . »

3. node2 — Worker (l'usine : application + base de données)

Attribut	Valeur
Rôle	worker (charges applicatives)
IP publique	63.180.201.231 (éphémère, change au redémarrage)
IP privée	10.2.50.190
Instance	i-0635cb49fcfc65063

Ce qui tourne dessus : - L'application **Laravel** (`laravel-app` , **2 réplicas**) — le cœur du projet, converti depuis `docker-compose` vers un **Helm chart**. - La base de données **MySQL** (`mysql-0` , StatefulSet) avec **stockage persistant** (PersistentVolume) et **sauvegardes** (CronJob). - Composants d'appoint : `coredns` (DNS interne), `metrics-server` (métriques pour l'auto-scaling), `argocd-repo-server` .

Pourquoi ce choix : on dédie un nœud aux **charges applicatives** pour respecter la bonne pratique de redondance : l'app tourne en **plusieurs réplicas** avec des **règles d'anti-affinité** (les réplicas évitent de tomber sur le même point de défaillance) et un **HPA** (auto-scaling 3→10 selon la charge CPU). La base de données a un **volume persistant** pour ne pas perdre les données au redémarrage d'un pod, et un **backup automatique** quotidien.

Points de démo : multi-réplicas + anti-affinité + HPA (montée en charge), persistance MySQL, sauvegarde à la demande.

4. node3 — Ingress (la porte d'entrée)

Attribut	Valeur
Rôle	ingress (routage du trafic externe)
IP publique	18.196.141.176 (éphémère)
IP privée	10.2.50.90
Instance	i-0fcdcb2880d2d6fb3

Ce qui tourne dessus : - `ingress-nginx-controller` : le reverse-proxy / load-balancer interne. Il reçoit le trafic HTTP(S) et le **route par nom d'hôte** vers le bon service (`larapp.kubequest.local` → l'app, `grafana.kubequest.local` → Grafana, etc.). - Interfaces d'administration exposées ici : **kubernetes-dashboard**, `argocd-server`. - Une partie de la stack logs (`loki`).

Pourquoi ce choix : on isole le **point d'entrée** du cluster sur un nœud dédié. Tout le trafic entrant passe par un seul contrôleur nginx qui applique le routage, la terminaison TLS (via cert-manager) et les règles d'authentification (SSO). Séparer l'ingress du control-plane est une **mesure de sécurité** : la surface exposée à Internet n'est pas la même machine que le cerveau du cluster.

Point de démo : un seul contrôleur nginx route plusieurs services par hostname ; c'est le seul point exposé.

5. node4 — Monitoring (la tour de contrôle : métriques + logs)

Attribut	Valeur
Rôle	monitoring (observabilité)
IP publique	3.67.67.127 (éphémère)
IP privée	10.2.50.113
Instance	i-05c12cfa80ed7e5e3

Ce qui tourne dessus : - **Prometheus** : collecte les métriques (CPU, RAM, requêtes...) de tout le cluster. - **Grafana** : visualise ces métriques (dashboards) + les **logs via Loki**. - **Alertmanager** : envoie les alertes (règles Prometheus). - **Loki** : agrège les logs de tous les pods (via l'agent **Promtail** qui tourne sur chaque nœud). - **kube-state-metrics** , **node-exporter** (métriques système par nœud).

Pourquoi ce choix : l'observabilité est gourmande (Prometheus stocke beaucoup de séries temporelles). On la met sur un nœud dédié pour qu'elle **ne concurrence pas l'application** en CPU/RAM, et pour centraliser « la tour de contrôle » du cluster à un seul endroit. Promtail, lui, est un **DaemonSet** : il tourne partout car il faut collecter les logs de chaque nœud, mais l'agrégation/visualisation est centralisée sur node4.

Point de démo : Grafana montre les métriques (Prometheus) ET les logs (Loki) au même endroit.

6. Ce qui est commun à TOUS les nœuds (DaemonSets)

Certains composants doivent tourner sur **chaque** nœud, par nature : - **calico-node** (CNI) : le réseau des pods, sur chaque machine. - **kube-proxy** : programme les règles réseau des Services sur chaque nœud. - **node-exporter** : expose les métriques système (CPU/RAM/disque) de chaque nœud à Prometheus. - **loki-promtail** : collecte les logs locaux de chaque nœud et les envoie à Loki.

7. Choix techniques transverses (le « pourquoi » global)

Choix	Justification
kubeadm (cluster non managé)	Le sujet demande de construire le cluster soi-même (pas d'EKS managé) → maîtrise complète du control-plane.
Calico (CNI)	Réseau pod-à-pod robuste + support des NetworkPolicies (isolation réseau, exigence sécurité).
Helm pour l'app	Packaging réutilisable, valeurs paramétrables, chart officiel pour la BDD.
Kustomize + ArgoCD (GitOps)	Déploiement déclaratif depuis Git ; ArgoCD réconcilie l'état → bonus GitOps.
Elastic IP sur node1 uniquement	Point d'administration stable ; les autres nœuds peuvent changer d'IP sans casser l'accès kubectl.
Rôles via labels + nodeSelector	Place chaque composant sur le bon nœud (ingress→node3, monitoring→node4...).
Extinction nocturne des VM	Contrainte AWS du sujet (coûts) → tout est redéployable via kubectl/helm/kustomize (exigence de défense).

8. Résumé express (à réciter en 30 secondes)

« Nous avons un cluster Kubernetes de 4 nœuds à responsabilités séparées : **node1** est le control-plane (le cerveau, avec etcd et l'API), **node2** exécute l'application Laravel en 2 répliques auto-scalables et sa base MySQL persistante, **node3** est la porte d'entrée avec le contrôleur nginx-ingress qui route tout le trafic, et **node4** est la tour de contrôle avec Prometheus, Grafana et Loki. Cette séparation isole les charges, réduit la surface d'attaque et rend l'exploitation lisible. »